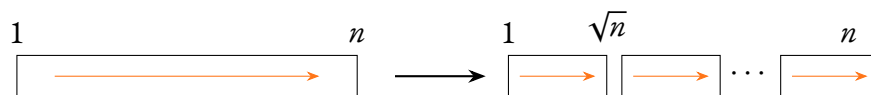


Lista 05 – Algoritmos e Estruturas de Dados (2026.1)

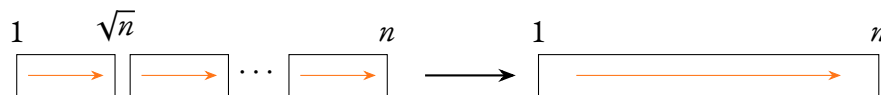
Tópico: Multilistas e Listas Esparsas

8 de maio de 2026

1. **Construção de multilista.** Apresente um algoritmo que construa uma multilista a partir de uma lista desordenada. Analise o tempo de execução do seu programa, assumindo que as ordenações são feitas com o algoritmo da bolha.



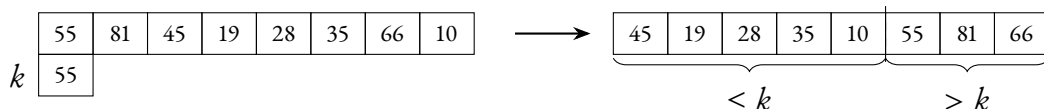
2. **Conversão de multilista para lista ordenada.** Faça um programa que converta uma multilista em uma lista completamente ordenada. *Dica: faça chamadas sucessivas ao procedimento de intercalação.* Também estime o tempo de execução do seu programa.



3. **Mediana por partição.** Considere novamente o problema da mediana: encontrar em uma lista $A[1 \dots n]$ o elemento que é maior que $n/2$ e menor ou igual a $n/2$ elementos da lista.

Uma solução trivial consiste em ordenar a lista e retornar o elemento da posição $n/2$. Mas é possível fazer algo mais inteligente.

A ideia é a seguinte. Aplicando o procedimento de partição sobre a lista, utilizando o primeiro elemento como pivô k , observamos que a mediana certamente está no maior lado da partição (ou do lado esquerdo, caso os dois lados tenham o mesmo tamanho). A ideia, então, é continuar a busca pela mediana fazendo uma nova partição no lado em que ela se encontra, reduzindo ainda mais a lista de candidatos, até que reste apenas a mediana.



- (a) Apresente um programa que implementa esse método para encontrar a mediana da lista.

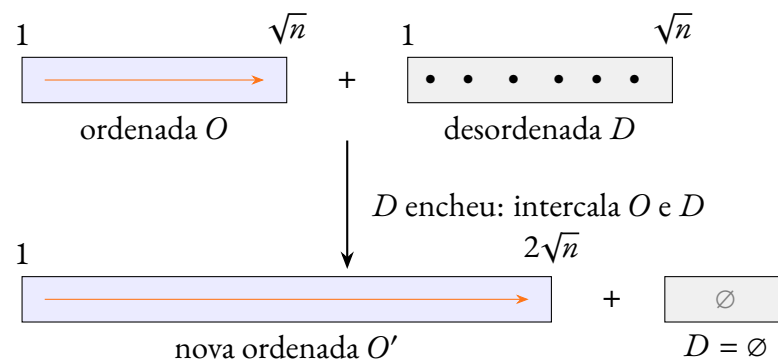
- (b) Assumindo que em cada etapa (aprox.) $1/4$ dos candidatos atuais são eliminados, estime o número de comparações realizadas pelo seu programa.
4. **Trade-off em multilistas.** Quando a lista de tamanho n é dividida em k partes iguais, os tempos das operações são aproximadamente:

$$\text{busca} : O(k \cdot \log(n/k)), \quad \text{inserção/remoção} : O(n/k).$$

- (a) Mostre que a escolha $k = \sqrt{n}$ equilibra os tempos de busca e de inserção/remoção, e derive as complexidades da tabela abaixo.
- (b) Para um sistema com muitas buscas e poucas inserções, qual deveria ser a estratégia de escolha de k ? Justifique.
5. **Estrutura híbrida ordenada/desordenada.** Considere uma estrutura de dados que mantém uma lista de n elementos dividida em duas partes de tamanho $\sqrt{n}$ cada:

- uma **parte ordenada** $O[1 \dots \sqrt{n}]$, mantida em ordem crescente;
- uma **parte desordenada** $D[1 \dots \sqrt{n}]$, usada como buffer de inserções.

Todo elemento da lista está em exatamente uma das duas partes. Quando a parte desordenada enche (i.e., atinge $\sqrt{n}$ elementos), ela é intercalada com a parte ordenada, produzindo uma nova parte ordenada de tamanho $2\sqrt{n} \approx \sqrt{2n}$, e a parte desordenada é esvaziada.

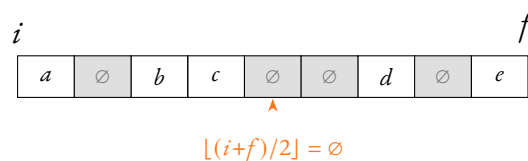


- (a) Descreva o algoritmo de **busca** por um elemento k nessa estrutura. Qual é o seu tempo de execução no pior caso?
- (b) Descreva o algoritmo de **inserção** de um elemento k . Qual é o tempo de execução de uma única inserção no pior caso? E o tempo **amortizado** por inserção, considerando uma sequência de n inserções a partir de uma estrutura vazia?

- (c) Descreva o algoritmo de **remoção** de um elemento k . Distinga os dois casos: elemento encontrado na parte desordenada e elemento encontrado na parte ordenada. Qual é o tempo de execução em cada caso?
- (d) Compare os tempos das três operações com os da multilista e com os da lista ordenada simples. Em que situação essa estrutura híbrida seria preferível às outras duas?
6. **Listas esparsas — inserção e remoção.** Considere uma lista esparsa com n posições totais, das quais 90% estão ocupadas e 10% estão vazias, distribuídas uniformemente ao longo da lista.



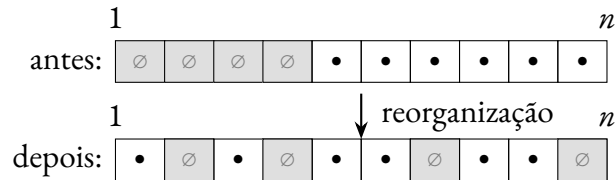
- (a) Explique por que a operação de remoção leva tempo $O(1)$ (desconsiderando a busca). O que é feito fisicamente?
- (b) Descreva as quatro etapas da operação de inserção na lista esparsa.
- (c) Com as hipóteses de distribuição uniforme das posições vazias, por que os passos de “localizar posição vazia” e “deslocar elementos” levam tempo $O(1)$?
- (d) O que acontece com o desempenho da lista esparsa quando as posições vazias se concentram em uma extremidade da lista? Como isso poderia ser evitado?
7. **Busca binária na lista esparsa.** Na lista esparsa, as posições vazias dificultam a aplicação direta da busca binária, pois a posição central pode estar vazia. A figura abaixo ilustra uma situação em que a posição central $\lfloor (i + f)/2 \rfloor$ é vazia e é preciso escolher um representante não-vazio próximo a ela.



- (a) Proponha uma adaptação do algoritmo de busca binária que lide corretamente com posições marcadas como vazias.
- (b) Analise o tempo de execução do seu algoritmo adaptado no melhor e no pior caso.

(c) A presença de posições vazias piora o tempo de busca em relação à lista ordenada tradicional? Justifique.

8. **Reorganização da lista esparsa.** Após uma sequência de remoções, as posições vazias podem se concentrar e degradar o desempenho. A figura mostra uma lista antes e depois da reorganização.



- Apresente um algoritmo de reorganização que redistribua uniformemente as posições vazias ao longo da lista, sem alterar a ordem dos elementos presentes.
- Analise o tempo de execução do seu algoritmo de reorganização.
- Proponha uma política (baseada em algum critério mensurável) para decidir o momento adequado de chamar esse algoritmo.